

Você conhece o Padrão Observer?

Vamos aprender um pouco sobre
este Design Pattern!

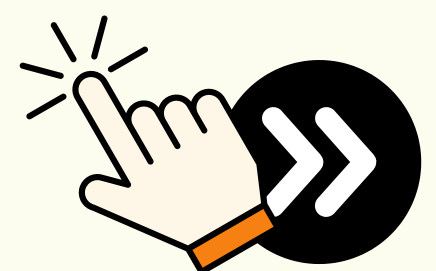
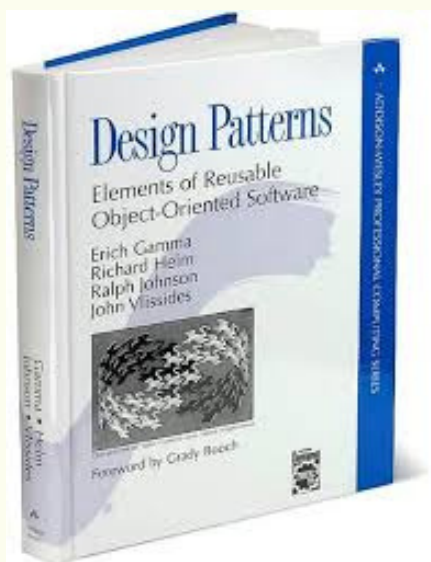




Mas afinal, o que é Observer?

É um padrão comportamental que define uma relação um-para-muitos entre objetos.

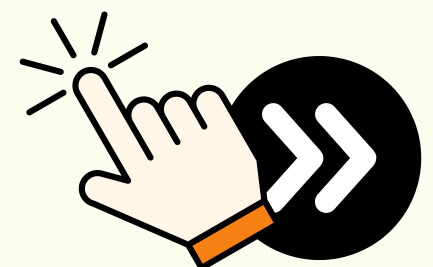
Quando um objeto muda de estado, todos os seus dependentes são notificados automaticamente.



O que ele resolve?

Ele desacopla o objeto que emite eventos (o Subject) dos objetos que reagem a esses eventos (os Observers).

 Ideal quando várias partes do sistema precisam reagir a uma mudança sem depender diretamente umas das outras.



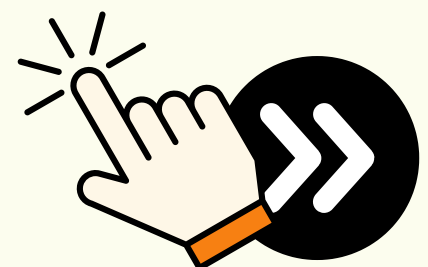
Exemplo

 Use Case: Criação de Usuário com Notificação por E-mail

 UserService: gerencia usuários

 EmailNotifier: envia e-mails quando um usuário é criado

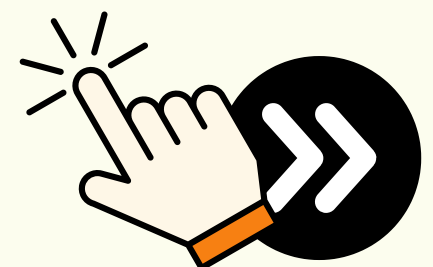
O EmailNotifier se registra no UserService, e é notificado automaticamente sempre que um novo usuário é criado.





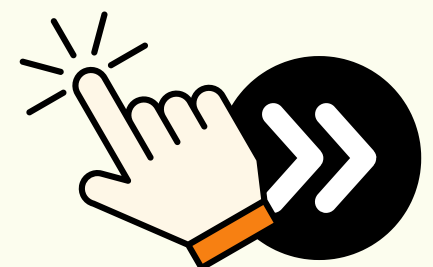
```
interface Observer {  
    void update(String data);  
}  
  
interface Subject {  
    void registerObserver(Observer o);  
    void notifyObservers(String data);  
}
```

 Interfaces básicas do padrão



```
class UserService implements Subject {  
    private List<Observer> observers = new ArrayList<>();  
  
    public void registerObserver(Observer o) {  
        observers.add(o);  
    }  
  
    public void notifyObservers(String data) {  
        for (Observer o : observers) {  
            o.update(data);  
        }  
    }  
  
    public void createUser(String name) {  
        // lógica de criação  
        System.out.println("Usuário criado: " + name);  
        notifyObservers(name);  
    }  
}
```

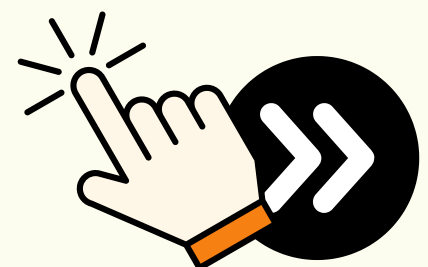
 Implementação prática em Java



In/devsjava

```
class EmailNotifier implements Observer {  
    public void update(String userName) {  
        System.out.println("Enviando e-mail de boas-vindas para: " + userName);  
    }  
}
```

 O Observer que envia e-mail

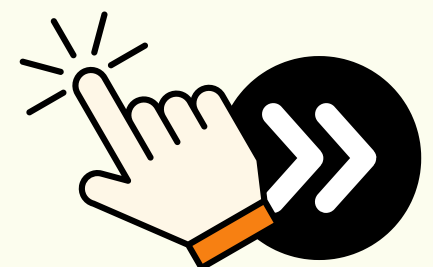




```
UserService userService = new UserService();  
EmailNotifier notifier = new EmailNotifier();  
  
userService.registerObserver(notifier);  
userService.createUser("Ana Silva");
```

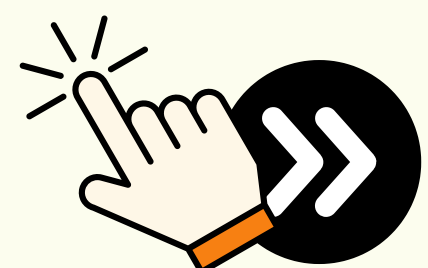
▶ Executando o fluxo completo

✓ Resultado:
Usuário criado
Notificação de boas-vindas enviada automaticamente



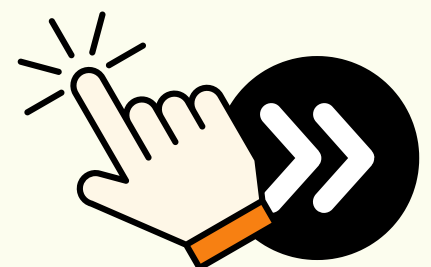
Vantagens do Observer

- **Baixo acoplamento** entre componentes
- **Fácil de extender:** novos observers podem ser adicionados sem alterar o código do Subject
- **Reatividade:** útil em interfaces gráficas, sistemas de eventos, logs, etc.



Desvantagens

- **Depuração** difícil quando há muitos observers
- **Ordem de execução não garantida**
- **Dependências implícitas** podem tornar o sistema difícil de entender e manter





Conclusão

O padrão Observer é ideal quando você quer notificar vários objetos de uma mudança de estado sem acoplamento direto.

Você já usou esse padrão em algum projeto?
Me conta nos comentários! 🙋

